

SAVCNG ExcelDNA - Documentación Técnica Completa

Versión: 2.4.0.0

Última Actualización: 11 de julio de 2026

Estado: En Desarrollo Continuo

Tabla de Contenidos

1. [Descripción General](#)
2. [Arquitectura del Proyecto](#)
3. [Componentes Principales](#)
4. [Sistema de Validaciones Detallado](#)
 - [4.1 Validación de Decimales \(Enteros\)](#)
 - [4.2 Validación de Catálogos](#)
 - [4.3 Validación NS](#)
 - [4.4 Validación de Formato Texto](#)
 - [4.5 Validación de Bloqueo Dinámico](#)
 - [4.6 Validación de Campos Vacíos \(Blancos\)](#)
 - [4.7 Validación de Especifique por Palabras Clave](#)
 - [4.8 Validación de Años \(Fechas\)](#)
 - [4.9 Validación de Sumas Cruzadas y Verticales](#)
5. [FrmValidaciones.cs - Análisis Profundo](#)
6. [Flujo de Trabajo](#)
7. [Especificaciones Técnicas](#)
8. [Patrones de Código y Mejores Prácticas](#)
9. [Troubleshooting y Casos Especiales](#)

Descripción General

SAVCNG ExcelDNA es un complemento (Add-in) de Microsoft Excel desarrollado en C# (.NET Framework 4.8) que proporciona un sistema avanzado de validación de datos para archivos de censo de gobierno.

El proyecto utiliza **ExcelDNA** como framework de integración con Excel, permitiendo crear un menú personalizado en la cinta de opciones (Ribbon) de Excel desde el cual se pueden cargar archivos de censo y aplicar múltiples validaciones de integridad de datos.

Propósito Principal

- Garantizar la calidad e integridad de los datos en archivos de censo
- Proporcionar una interfaz gráfica intuitiva para validación de datos
- Detectar y resaltar errores de formato y contenido
- Facilitar listas desplegables (Data Validation) personalizadas
- Validar presencia de respuestas "No Aplica" (NS) con alertas contextuales

- Validar formato de texto: mayúsculas, sin espacios extra, caracteres válidos
- Crear reglas de bloqueo dinámico condicionadas a valores de otras celdas
- Soporte automático para matrices paralelas y evaluaciones fila por fila
- Mensajes de alerta personalizados que aparecen dinámicamente
- Detectar y resaltar campos vacíos en matrices con validación de obligatoriedad
- Detectar si el texto de un "Especifique" coincide con opciones del catálogo mediante palabras clave
- Validar que el año capturado se encuentre dentro de un rango histórico configurable
- Verificar consistencia de sumas horizontales e inyectar sumatorias verticales automáticas

Tecnologías Utilizadas

- **Lenguaje:** C# 7.3
- **Framework:** .NET Framework 4.8
- **Librería de Interop:** Microsoft.Office.Interop.Excel
- **Plugin Framework:** ExcelDNA
- **UI:** Windows Forms (.NET Framework)

Arquitectura del Proyecto

```
SAVCNG_ExcelDNA/
├── MenuCenso.cs           (Punto de entrada y menú)
├── FrmValidaciones.cs     (Lógica principal de validaciones)
├── FrmValidaciones.Designer.cs (Diseño de formulario)
├── Properties/
│   └── AssemblyInfo.cs    (Metadatos del ensamblado)
├── bin/                   (Ejecutables compilados)
├── obj/                   (Artefactos compilados)
└── documentacion/
    └── documentacion_savcng_YYYY-MM-DD.md (Documentación técnica)
```

Patrón Arquitectónico

- **Model:** Las colecciones (List, Dictionary) almacenan datos de validación y rangos
- **View:** Windows Forms (FrmValidaciones) para la interfaz de usuario
- **Controller:** Métodos de eventos que procesan interacciones del usuario
- **Service:** Métodos estáticos y funciones auxiliares para lógica compartida

Componentes Principales

1. **MenuCenso.cs** - Punto de Entrada

Responsable de la integración con Excel y la interfaz de usuario del ribbon.

Funcionalidades:

- **XML Ribbon:** Define la estructura del menú en la pestaña "SAVCNG - Censo"
- **Diálogo de Archivo:** Permite seleccionar archivos de censo (.xlsx, .xlsm, .xlsb)
- **Gestión de Workbook:** Abre el archivo y lo pasa a la ventana de validaciones
- **Manejo de Errores:** Captura excepciones durante la carga de archivos

Flujo de Ejecución:

1. Usuario hace clic en "Cargar Censo" en el menú de Excel
2. Se abre un diálogo `OpenFileDialog`
3. El archivo seleccionado se abre en Excel mediante `excelApp.Workbooks.Open()`
4. Se instancia `FrmValidaciones` pasando el workbook
5. La ventana se muestra flotante sobre Excel

2. FrmValidaciones.cs - Motor de Validaciones

Este es el corazón del aplicativo. Implementa todas las validaciones de datos y maneja la interacción del usuario.

Variables de Estado Principales

```
private Excel.Workbook _libroCenso;           // Archivo de Excel cargado
private Excel.Range _rangoCapturado;        // Rango seleccionado por el usuario
private Excel.Range _rangoCatalogo;         // Rango de opciones del catálogo
private List<Excel.Range> _pendientesCatalogo;
private Dictionary<string, string> _preguntaPorRango;
```

Sistema de Validaciones Detallado

4.1 Validación de Decimales (Enteros)

Objetivo: Prevenir que el usuario capture números con decimales o texto cuando la celda solo admite enteros.

Tipo: Data Validation - Custom (Restictiva)

Restricción: Sí (lanza ventana emergente si se viola)

Motor: `O(ESBLANCO()); Y(ESNUMERO(); TRUNCAR(=valor))`

Fórmula:

```
=O(ESBLANCO(A1); Y(ESNUMERO(A1); TRUNCAR(A1)=A1))
```

Mensaje de Error:

Título: Captura Inválida (Solo Enteros)
Mensaje: El formato de esta celda no admite números con decimales ni texto.
Por favor, introduce únicamente un número entero (Ej: 1, 15, 100).

Implementación:

```
Excel.Range primeraCelda = (Excel.Range)_rangoCapturado.Cells[1, 1];  
string direccion = primeraCelda.Address;  
  
string formulaDecimales =  
    $"=O(ESBLANCO({direccion}){separador}Y(ESNUMERO({direccion})  
{separador}TRUNCAR({direccion})={direccion}))";  
  
_rangoCapturado.Validation.Add(  
    Excel.XlDVType.xlValidateCustom,  
    Excel.XlDValertStyle.xlValidAlertStop,  
    Excel.XlFormatConditionOperator.xlBetween,  
    formulaDecimales,  
    Type.Missing);
```

4.2 Validación de Catálogos

Objetivo: Crear listas desplegables (Data Validation) con opciones predefinidas.

Tipo: Data Validation - Lista

Restricción: Sí (solo permite valores de la lista)

Motor: Rango de opciones o entrada manual

Características:

- Entrada manual de opciones separadas por comas (ej: "1,2,9" o "X")
- Selección de celdas con tres modos: Solo Números, Texto Exacto, Referencia de Rango
- Soporte para celda única, celdas combinadas y rangos múltiples
- Limpieza automática de caracteres innecesarios

4.3 Validación NS (No Aplica)

Objetivo: Permitir números ≥ 0 o valor "NS", con alerta dinámica cuando se registra NS.

Tipo: Data Validation - Custom + Fórmula de alerta

Restricción: Sí (solo permite números ≥ 0 o "NS")

Motor: `O(Y(ESNUMERO()); >=0); ="NS")`

Fórmula de Validación (español con separador dinámico):

```
=O(Y(ESNUMERO(A1); A1>=0); A1="NS")
```

Fórmula de Alerta (inglés universal):

```
=IF(SUM(COUNTIF($A$1:$A$10,"NS"), COUNTIF($B$1:$B$10,"NS"))>0, "Alerta...", "")
```

4.4 Validación de Formato Texto

Objetivo: Garantizar texto en mayúsculas, sin espacios extra y sin caracteres especiales.

Tipo: Data Validation - Custom con Rango Auxiliar

Restricción: Sí (previene entrada inválida)

Motor: Fórmula binaria en columna auxiliar + `= {celdaAux}=1`

Whitelist de Caracteres Permitidos:

```
0123456789ABCDEFGHIJKLMNÑOPQRSTUVWXYZÁÉÍÓÜÜ (espacio)
```

Fórmula Auxiliar (binaria, inglés universal):

```
=IF(OR(ISBLANK(A1),  
    AND(EXACT(A1, UPPER(A1)),  
        LEN(A1)=LEN(TRIM(A1))),  
    SUMPRODUCT(--  
    ISNUMBER(FIND(MID(A1,ROW(INDIRECT("1:"&MAX(1,LEN(A1))))),1),"0123456789ABC..."))=LEN  
(A1)  
)), 1, 0)
```

4.5 Validación de Bloqueo Dinámico

Objetivo: Crear reglas de bloqueo condicional basadas en valores de otras celdas, con soporte para matrices paralelas.

Tipo: Data Validation - Custom + Formatos Condicionales

Restricción: Sí (bloquea entrada si condición no se cumple)

Motor: `CONTAR.SI()` con detección inteligente de modo fila-por-fila o global

Jerarquía de Formatos Condicionales:

```
Regla 1 – Gris (Bloqueado): StopIfTrue = true  
Regla 2 – Azul (Obligatorio): StopIfTrue = true
```

4.6 Validación de Campos Vacíos (Blancos)

Objetivo: Detectar y resaltar celdas vacías en filas que ya tienen datos en otras columnas.

Tipo: Fórmula de alerta + Formato Condicional

Restricción: No (solo resalta visualmente)

Motor: `SUMPRODUCT()` con álgebra booleana fila-por-fila

Fórmula de Alerta Global (inglés universal):

```
=IF(SUMPRODUCT(($M$52:$M$71<>"")+($N$52:$N$71<>""))*  
    (($M$52:$M$71=""")+($N$52:$N$71="""))>0,  
    "Favor de revisar información faltante", "")
```

4.7 Validación de Especifique por Palabras Clave ★ (v2.3.0)

Objetivo: Detectar si el texto libre capturado en un campo "Especifique" coincide semánticamente con alguna de las opciones de un catálogo.

Tipo: Motor Auxiliar Matricial + Formato Condicional + Alerta

Restricción: No (es informativa)

Motor: Diccionario de palabras clave con `SEARCH()` + `OR()` nativo de Excel

Truco Arquitectónico — Celda Dummy:

```
Excel.Range celdaDummy = ws.Cells[1048576, 16384];  
celdaDummy.Formula = formulaFormatCondIngles;  
string formulaFormatCondLocal = celdaDummy.FormulaLocal;  
celdaDummy.Clear();
```

4.8 Validación de Años (Fechas) ★ (v2.3.0)

Objetivo: Restringir la captura a años numéricos dentro de un rango histórico configurable, con soporte adicional para el código "NS".

Tipo: Data Validation - Custom

Restricción: Sí

Motor: `O(ESPACIOS()="NS"; Y(ESNUMERO()); >=LimInf; <=LimSup))`

Fórmula de Validación:

```
=O(ESPACIOS(A1)="NS"; Y(ESNUMERO(A1); A1>=1821; A1<=2026))
```

4.9 Validación de Sumas Cruzadas y Verticales (v2.4.0)

Objetivo: Verificar que la suma horizontal de los desagregados coincida con la columna de total fila a fila, y calcular automáticamente las sumatorias verticales de la tabla.

Tipo: Formato Condicional (expresión booleana) + Fórmulas `SUM()` automáticas

Restricción: No (resalta visualmente el error; no impide la captura)

Motor: Álgebra booleana `AND(ISNUMBER; COUNTIF NS=0; Total <> Suma desagregados)`

Contexto de Uso:

Tabla de captura:

Col. Total	Des. A	Des. B	Des. C	
100	30	40	20	← ROJO: 30+40+20=90 ≠ 100
90	30	40	20	← OK
SUM↓	SUM↓	SUM↓	SUM↓	← Fórmulas verticales inyectadas

Flujo de Configuración (3 pasos):

- Paso 1:** Seleccionar la columna del **Total** (una columna, mismas filas del rango capturado)
- Paso 2:** Seleccionar las columnas de los **Desagregados** (admite columnas no contiguas con `Ctrl`)
- Paso 3:** Seleccionar la fila/celdas destino de la **Sumatoria Vertical (Σ)** al pie de la tabla

Fases de Implementación:

Fase 1 — Análisis Espacial:

```
int filaInicio = _rangoCapturado.Row;
int filaFin    = filaInicio + _rangoCapturado.Rows.Count - 1;
```

```
Excel.Range celdaTotalAux = (Excel.Range)rangoTotal.Cells[1, 1];  
string letraTotal = celdaTotalAux.Address.Split('$')[1];
```

Fase 2 — Extracción de Letras de Columna (COM-safe):

Itera de forma segura sobre todas las áreas de `rangoDesagregados`, haciendo cast explícito antes de acceder a `.Address`:

```
foreach (Excel.Range area in rangoDesagregados.Areas)  
{  
    for (int c = 1; c <= area.Columns.Count; c++)  
    {  
        Excel.Range colCelda = (Excel.Range)area.Cells[1, c];  
        string letra = colCelda.Address.Split('$')[1];  
  
        if (!letrasDesagregados.Contains(letra))  
        {  
            letrasDesagregados.Add(letra);  
            fragmentosExclusionNS.Add($"COUNTIF({letra}{filaInicio},\"NS\")=0");  
            fragmentosExclusionNS.Add($"COUNTIF({letra}{filaInicio},\"ns\")=0");  
        }  
        Marshal.ReleaseComObject(colCelda);  
    }  
}
```

Fase 3 — Construcción de la Fórmula de Error (inglés universal):

La fórmula es fila-sensible (referencias relativas en fila, absolutas en columna):

```
=AND(  
    ISNUMBER($T{fila}),  
    COUNTIF($T{fila}, "NS")=0, COUNTIF($T{fila}, "ns")=0,  
    COUNTIF($A{fila}, "NS")=0, COUNTIF($A{fila}, "ns")=0,  
    ...  
    $T{fila} <> ($A{fila} + $B{fila} + $C{fila})  
)
```



```

string formulaSumandos = string.Join("+", letrasDesagregados.ConvertAll(1 => $"${1}
{filaInicio}"));
string formulaCondicionesNS = string.Join(",", fragmentosExclusionNS);
string formulaErrorFilaIngles =
    $"=AND(ISNUMBER({letraTotal}{filaInicio}), {formulaCondicionesNS}, " +
    $"${letraTotal}{filaInicio}<>({formulaSumandos}))";

```

Fase 4 — Traducción Nativa Fila-Sensible (Truco Arquitectónico):

La celda dummy se coloca en la **misma fila de inicio** del rango capturado (columna XFD / 16384). Esto garantiza que las referencias relativas de fila no se desfasen al leer `FormulaLocal`:

```

Excel.Worksheet wsActual = (Excel.Worksheet)_rangoCapturado.Worksheet;
celdaDummy = (Excel.Range)wsActual.Cells[filaInicio, 16384];

celdaDummy.Formula = formulaErrorFilaIngles;
string formulaErrorFilaLocal = celdaDummy.FormulaLocal;
celdaDummy.Clear();
Marshal.ReleaseComObject(celdaDummy);
celdaDummy = null;

```

Fase 5 — Auditoría de Coexistencia:

Antes de inyectar, detecta si ya existen formatos condicionales previos (por ejemplo, de otra capa de Sumas en matrices jerárquicas con subtotales):

```

if (_rangoCapturado.FormatConditions.Count > 0)
{
    DialogResult respFormato = MessageBox.Show(this,
        "Se detectaron reglas de validación de sumas previas en este rango.\n\n" +
        "¿Deseas CONSERVARLAS e integrar esta nueva capa de revisión?\n\n" +
        "Sí = Apilar reglas (Recomendado para matrices jerárquicas con
subtotales).\n\n" +
        "NO = Borrar las reglas anteriores y dejar solo esta nueva.",
        "SAVCNG - Formatos Condicionales Detectados",
        MessageBoxButtons.YesNoCancel, MessageBoxIcon.Warning);

    if (respFormato == DialogResult.Cancel) { return; }
    if (respFormato == DialogResult.No) { _rangoCapturado.FormatConditions.Delete(); }
}

```

Fase 6 — Inyección del Formato Condicional:

Aplica la fórmula traducida como `xlExpression`. El color de error es Rojo Claro con fuente Rojo Oscuro. `StopIfTrue = false` para permitir apilamiento multinivel:

```
Excel.FormatCondition fcError =
(Excel.FormatCondition)_rangoCapturado.FormatConditions.Add(
    Excel.XlFormatConditionType.xlExpression, Type.Missing, formulaErrorFilaLocal);

fcError.Interior.Color = ColorTranslator.ToOle(Color.FromArgb(255, 199, 206)); //
Rojo Claro
fcError.Font.Color      = ColorTranslator.ToOle(Color.FromArgb(156, 0, 6));      //
Rojo Oscuro
fcError.StopIfTrue      = false;
Marshal.ReleaseComObject(fcError);
```

Fase 7 — Inyección de Sumatorias Verticales:

Itera sobre cada celda del rango destino seleccionado en el Paso 3 y escribe la fórmula `SUM` correspondiente, liberando cada objeto COM inmediatamente:

```
foreach (Excel.Range celdaSumatoria in rangoTotalesVerticales.Cells)
{
    Excel.Range celdaSumatoriaAux = (Excel.Range)celdaSumatoria;
    string letraColVertical = celdaSumatoriaAux.Address.Split('$')[1];
    celdaSumatoriaAux.Formula = $"=SUM({letraColVertical}{filaInicio}:
{letraColVertical}{filaFin})";
    Marshal.ReleaseComObject(celdaSumatoriaAux);
}
```

Gestión de Memoria:

```
finally
{
    if (celdaTotalAux != null) Marshal.ReleaseComObject(celdaTotalAux);
    if (celdaDummy != null) Marshal.ReleaseComObject(celdaDummy);
    if (rangoTotal != null) Marshal.ReleaseComObject(rangoTotal);
    if (rangoDesagregados != null) Marshal.ReleaseComObject(rangoDesagregados);
    if (rangoTotalesVerticales != null)
        Marshal.ReleaseComObject(rangoTotalesVerticales);
}
```

Comportamiento Visual:

Estado de la fila	Color aplicado
Total $\neq$ Suma de desagregados (y todos son numéricos y sin NS)	Fondo Rojo Claro #FFC7CE, Fuente Rojo Oscuro #9C0006
Total = Suma de desagregados	Sin color (celda normal)
Cualquier celda contiene "NS" o "ns"	Sin color (excluida de la regla matemática)
Celda del total no es número	Sin color (excluida de la regla matemática)

Ventajas:

- ✓ Funciona con rangos de desagregados no contiguos (columnas separadas)
- ✓ Excluye automáticamente filas con "NS" para no generar falsos positivos
- ✓ Soporta apilamiento multinivel (matrices con subtotales y totales generales)
- ✓ Inyecta las sumatorias verticales automáticamente sin intervención manual
- ✓ Truco de celda dummy en la misma fila para traducción fila-sensible precisa
- ✓ Gestión COM completa con `Marshal.ReleaseComObject`
- ✓ `StopIfTrue = false` para coexistencia con otras reglas de formato condicional

FrmValidaciones.cs - Análisis Profundo

5.1 Método: btnCapturarRango_Click

```

if (seleccion is Excel.Range)
{
    _rangoCapturado = (Excel.Range)seleccion;
    string pregunta = ObtenerNumeroPregunta(_rangoCapturado);
    lblPregunta.Text = "Pregunta detectada: " + pregunta;
    lblRangoSeleccionado.Text = "Rango seleccionado: " +
        _rangoCapturado.Address.Replace("$", "");

    MessageBox.Show(this, "Se capturó correctamente el rango: " +
        _rangoCapturado.Address, "Captura exitosa",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
}

```

5.2 Método: btnAplicar_Click - Estructura General v2.4.0

```

btnAplicar_Click()
|
|— Validación previa: _rangoCapturado != null
|

```

```

└─ if (chkDecimales.Checked)
    └─ Validación de Enteros – Data Validation restrictiva

└─ else if (chkCatalogos.Checked)
    └─ Validación de Catálogos – Manual o Seleccionar celdas

└─ else if (chkNS.Checked)
    └─ Validación NS – Destino y texto configurables

└─ else if (chkFormatoTexto.Checked)
    └─ Validación Formato Texto – Motor auxiliar + Whitelist

└─ else if (chkBloqueo.Checked)
    └─ Validación de Bloqueo Dinámico – 4 pasos + auditoría

└─ else if (chkBlancos.Checked)
    └─ Validación de Campos Vacíos – Motor SUMPRODUCT + auditoría

└─ else if (chkEspClave.Checked)
    └─ Validación Especifique por Palabras Clave
        └─ Fase 1: Celda de limpieza (normalización)
        └─ Fase 2: Motor de palabras clave por opción
        └─ Fase 3: Traducción de fórmulas (celda dummy)
        └─ Fase 4: Mensaje de alerta global

└─ else if (chkFechas.Checked)
    └─ Validación de Años
        └─ Captura de límite inferior y superior
        └─ Validación numérica de entradas
        └─ Fórmula: O(ESPACIOS="NS"; Y(ESNUMERO; >=; <=))

└─ else if (chkSumas.Checked) ★ NUEVA v2.4.0
    └─ Validación de Sumas Cruzadas y Verticales
        └─ Paso 1: Captura de columna Total
        └─ Paso 2: Captura de columnas Desagregados (no contiguas admitidas)
        └─ Paso 3: Captura de fila destino Sumatoria Vertical
        └─ Fase 1: Análisis espacial (filaInicio, filaFin, letraTotal)
        └─ Fase 2: Extracción COM-safe de letras de columna + exclusión NS
        └─ Fase 3: Construcción de fórmula booleana en inglés universal
        └─ Fase 4: Traducción nativa fila-sensible (celda dummy en XFD/filaInicio)
        └─ Fase 5: Auditoría de coexistencia (apilamiento o limpieza)
        └─ Fase 6: Inyección del FormatCondition (Rojo Claro/Oscuro,
StopIfTrue=false)
        └─ Fase 7: Inyección de fórmulas SUM verticales automáticas

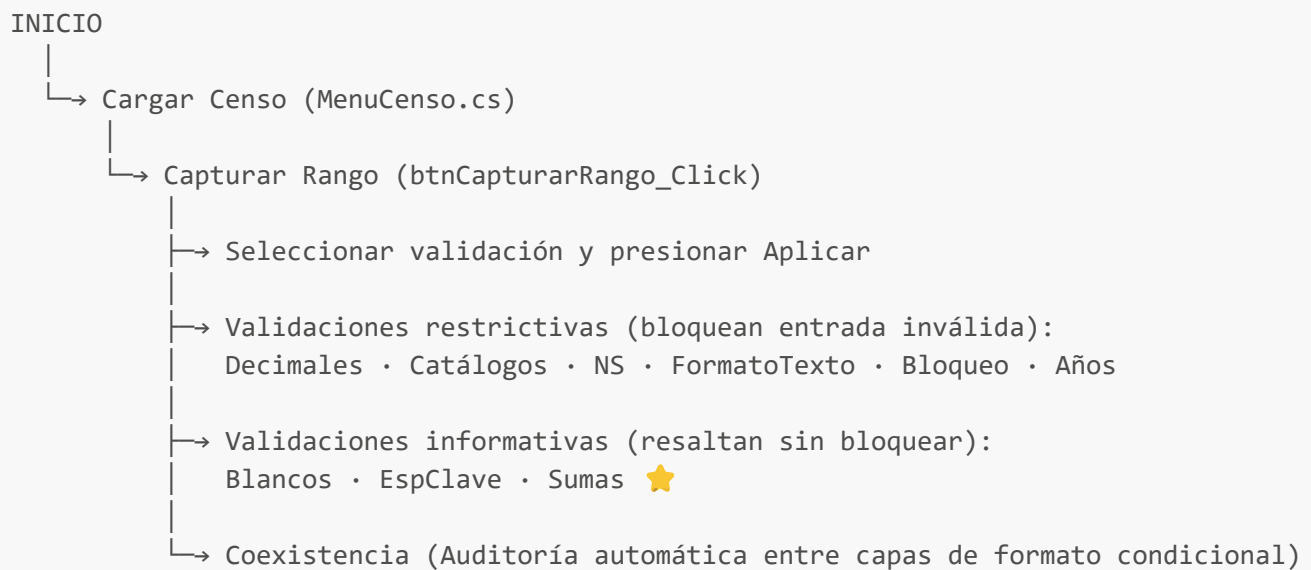
```

5.3 Métodos CheckedChanged

Todos los checkboxes siguen el mismo patrón de validación previa:

```
private void chkXxx_CheckedChanged(object sender, EventArgs e)
{
    if (chkXxx.Checked)
    {
        if (_libroCenso == null || _rangoCapturado == null)
        {
            MessageBox.Show(this, "Primero carga un censo y captura un rango.",
                "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
            chkXxx.Checked = false;
        }
    }
}
```

Flujo de Trabajo v2.4.0



Especificaciones Técnicas

Requisitos del Sistema

Aspecto	Requisito
Versión .NET	.NET Framework 4.8
Versión C#	7.3
Versión Excel	2013 o superior
SO	Windows 7/8/10/11 (32 o 64 bits)

Aspecto	Requisito
Memoria RAM Mínima	4 GB
Bitness	Mismo que Office instalado (32 o 64)

Dependencias

```
Microsoft.Office.Interop.Excel (versión 15.0+)
ExcelDna.Integration
System.Windows.Forms
System.Drawing
System.Text.RegularExpressions
System.Reflection
System.Collections.Generic
System.Runtime.InteropServices
```

Internacionalización

Español	Inglés	Función
ESNUMERO()	ISNUMBER()	Verificar si es número
TRUNCAR()	TRUNC()	Truncar decimales
CONTAR.SI()	COUNTIF()	Contar con condición
SUMPRODUCT()	SUMPRODUCT()	Suma de productos
SI()	IF()	Condicional
O()	OR()	O lógico
Y()	AND()	Y lógico
MAYUSC()	UPPER()	Convertir a mayúsculas
ESPACIOS()	TRIM()	Remover espacios extra
IGUAL()	EXACT()	Comparación exacta
LARGO()	LEN()	Largo de texto
ESBLANCO()	ISBLANK()	Verificar si está vacío
BUSCAR()	SEARCH()	Buscar texto
SUMA()	SUM()	Sumar rango

Separador dinámico:

```
string separador = excelApp.International[
    Excel.XlApplicationInternational.xlListSeparator].ToString();
```

Rendimiento v2.4.0

Operación	Tiempo Estimado
Captura de rango	Inmediato
Validación Decimales	1-2 seg
Validación Catálogos	2-3 seg
Validación NS	3-5 seg
Validación Formato Texto	2-4 seg
Validación Bloqueo completa	10-15 seg
Validación Blancos matriz 50×7	5-8 seg
Validación EspClave (catálogo 20 opciones)	5-10 seg
Validación Años	1-2 seg
Validación Sumas (matriz 50 filas, 5 columnas)	2-4 seg

Límites Conocidos

- Máximo rango: 1,048,576 filas × 16,384 columnas
- Máximo opciones catálogo: ~256 opciones
- Máximo formatos condicionales: 3 reglas simultáneas recomendadas (Sumas admite apilamiento)
- Máximo texto mensaje: 255 caracteres
- Máximo opciones EspClave: ~50 opciones (por rendimiento)
- Máximo columnas desagregados Sumas: ~50 (límite práctico)

Patrones de Código y Mejores Prácticas

Pattern 1: MessageBox con propietario

```
MessageBox.Show(this, "Mensaje", "Título", MessageBoxButtons.OK,
    MessageBoxIcon.Information);
```

Pattern 2: Casting Explícito (COM Interop)

```
Excel.Range primeraCelda = (Excel.Range)_rangoCapturado.Cells[1, 1];  
string direccion = primeraCelda.Address;
```

Pattern 3: Gestión de Memoria COM

```
Excel.Range obj = null;  
try  
{  
    obj = (Excel.Range)excelApp.InputBox(..., 8);  
}  
finally  
{  
    if (obj != null) Marshal.ReleaseComObject(obj);  
}
```

Pattern 4: Fórmulas Universales en INGLÉS

```
rangoDestino.Formula = "=IF(COUNTIF($A$1:$A$10,\"NS\")>0,\"Alerta\",\"\")";
```

Pattern 5: Álgebra Booleana Universal

```
string formula = $"=({celdaVacía}=\"\")*(({sumaActivadores}>0))";
```

Pattern 6: Bypass de Error 0x800A03EC

```
bool estabaVacía = (primeraCeldaCap.Value2 == null);  
if (estabaVacía) primeraCeldaCap.Value2 = "A";  
// ... inyectar validación ...  
if (estabaVacía) primeraCeldaCap.Value2 = null;
```

Pattern 7: Traducción de Fórmulas vía Celda Dummy

```
Excel.Range celdaDummy = ws.Cells[1048576, 16384]; // Última celda XFD1048576  
celdaDummy.Formula = formulaEnIngles;  
string formulaLocal = celdaDummy.FormulaLocal;  
celdaDummy.Clear();
```


Pattern 8: Celda Dummy Fila-Sensible ★ (v2.4.0)

Variante del Pattern 7 para fórmulas con referencias relativas de fila. La celda dummy se ubica en la **misma fila de inicio** del rango para que `FormulaLocal` devuelva referencias alineadas correctamente:

```
// CORRECTO: misma fila que el rango → referencias relativas de fila sin desfase
celdaDummy = (Excel.Range)wsActual.Cells[filaInicio, 16384];
celdaDummy.Formula = formulaConReferenciasRelativas;
string formulaLocal = celdaDummy.FormulaLocal;
celdaDummy.Clear();

// INCORRECTO (Pattern 7 clásico): fila 1048576 → puede desfasar referencias de fila
// celdaDummy = ws.Cells[1048576, 16384];
```

Troubleshooting y Casos Especiales

Problema 1: Error '`System.__ComObject`' no contiene definición para '`Address`'

Causa: Acceder a propiedades de un objeto COM sin hacer casting explícito.

Solución:

```
Excel.Range celda = (Excel.Range)rango.Cells[1, 1];
string dir = celda.Address;
```

Problema 2: Error `0x800A03EC` al inyectar Data Validation

Causa: La primera celda del rango está vacía al momento de crear la validación.

Solución: Aplicar el bypass temporal de valor "A" antes de la inyección.

Problema 3: Fórmula muestra `#¿NOMBRE?` en Excel español

Causa: Se inyectó una función en inglés mediante `.FormulaLocal`.

Solución: Usar siempre `.Formula` o el truco de la celda dummy para traducción automática.

Problema 4: Formato condicional desaparece al aplicar segunda validación

Causa: Se llama a `FormatConditions.Delete()` sin preguntar al usuario.

Solución: Implementar la auditoría de coexistencia antes de borrar.

Problema 5: `SUMPRODUCT` devuelve `#N/D`

Causa: Los vectores dentro de SUMPRODUCT tienen tamaños distintos.

Solución: Iterar columna-por-columna construyendo vectores del mismo tamaño.

Problema 6: EspClave muestra #¿NOMBRE? en el formato condicional

Causa: FormatConditions.Add no acepta funciones en inglés directamente en Excel español.

Solución: Usar el truco de la celda dummy (XFD1048576) para obtener la fórmula traducida.

Problema 7: Sumas — La regla de error se activa en filas con "NS" ★ (v2.4.0)

Causa: La operación aritmética Total <> Suma(desagregados) evalúa "NS" como 0, generando un falso positivo.

Solución: Los fragmentos COUNTIF(...,"NS")=0 y COUNTIF(...,"ns")=0 en la fórmula AND() excluyen cualquier fila donde el total o algún desagregado contenga "NS" o "ns".

Problema 8: Sumas — Referencias de fila desfasadas en la fórmula traducida ★ (v2.4.0)

Causa: Si la celda dummy se ubica en la fila 1048576 y la fórmula contiene referencias relativas de fila (ej: \$T5), FormulaLocal devuelve \$T1048576 en lugar de \$T5.

Solución: Colocar la celda dummy en la misma fila de inicio del rango capturado (wsActual.Cells[filaInicio, 16384]).

Resumen de Cambios v2.4.0

✦ Nueva Validación

Validación de Sumas Cruzadas y Verticales (chkSumas)

- Motor booleano AND(ISNUMBER; COUNTIF NS=0; Total <> Suma) en inglés universal
- Soporte para rangos de desagregados no contiguos (columnas separadas con Ctrl)
- Exclusión automática de filas con "NS"/"ns" para evitar falsos positivos
- Apilamiento multinivel con auditoría de coexistencia (matrices con subtotales)
- Inyección automática de fórmulas SUM verticales en la fila destino indicada
- Truco de celda dummy fila-sensible (Pattern 8) para traducción precisa
- StopIfTrue = false para coexistencia con otras capas de formato condicional
- Gestión completa de memoria COM con Marshal.ReleaseComObject
- Visualización: Fondo Rojo Claro RGB(255,199,206), Fuente Rojo Oscuro RGB(156,0,6)

🎨 Matriz de Validaciones Disponibles (v2.4.0)

#	Validación	Checkbox	Tipo	Restricción	Versión
1	Decimales (Enteros)	chkDecimales	Data Validation Custom	Sí	v2.3.0

#	Validación	Checkbox	Tipo	Restricción	Versión
2	Catálogos	chkCatalogos	Data Validation List	Sí	v2.2.2
3	NS	chkNS	Data Validation + Fórmula	Sí	v2.3.0
4	Formato Texto	chkFormatoTexto	Data Validation + Auxiliar	Sí	v2.3.0
5	Bloqueo Dinámico	chkBloqueo	Data Validation + Formatos	Sí	v2.2.4
6	Campos Vacíos	chkBlancos	Formato + Fórmula	No	v2.2.4
7	Especifique Clave	chkEspClave	Motor Auxiliar + Formato	No	v2.3.0
8	Años	chkFechas	Data Validation Custom	Sí	v2.3.0
9	Sumas Cruzadas	chkSumas	Formato Condicional + SUM	No ★	v2.4.0 ★

Conclusión

SAVCNG ExcelDNA en versión 2.4.0 alcanza nueve tipos de validación complementarios. La nueva validación de Sumas Cruzadas y Verticales cierra el ciclo de integridad cuantitativa: ya no solo se restringe el formato de los datos, sino que se verifica activamente que las sumas horizontales sean consistentes y que las sumatorias verticales queden calculadas de forma automática.

Fortalezas de v2.4.0: ✓ Nueve tipos de validación complementarios

- ✓ Restricciones preventivas e informativas según el tipo de error
- ✓ Coexistencia y apilamiento multinivel entre validaciones
- ✓ Gestión de memoria COM con `Marshal.ReleaseComObject`
- ✓ Compatibilidad universal multiidioma (`.Formula` en inglés)
- ✓ Interfaz completamente guiada por pasos
- ✓ Verificación activa de consistencia matemática fila a fila

Recomendaciones para v3.0:

- Historial de auditoría de validaciones aplicadas
- Exportar/importar configuraciones en JSON
- Vista previa de fórmulas antes de aplicar
- Undo/Redo de validaciones
- Validaciones de fecha completa (día/mes/año)
- Caché de configuraciones frecuentes
- Soporte para validación de sumas con tolerancia configurable (redondeo)

Versión: 2.4.0.0

Última Actualización: 11 de julio de 2026

Estado: En Desarrollo Continuo

Rama de Git: Desarrollo